

CS-550 PA-1
Fatima Mariyam – A20527616
Design Document

Overview

This assignment is a client-server mode file downloading system. It involves the creation of a client and server program. The server hosts a list of files for the client to download. The client will get the file list from the server and then request to download one or more files from the server. Scale the implementation of a client to evaluate the performance of the server.

Source Code

- Server_FS.java - server logic for accepting client connections and sending download data.
- Client_FS.java - client logic for downloading files.
- Utility.java – utility class for logging.
- SocketCnnectionPool.java – class for configuring socket connection pool.
- config.properties – properties file for project configurations
- Makefile
- scripts –
 - generate_files.sh – script to generate experiment data at the server location.
 - run_experiment_3.sh – script to run (3) of the problem statement.
 - run_experiment_4.sh – script to run (4) of the problem statement.
- Readme

Configurable parameters

Following are the parameters with their default values, that can be configured through the *config.properties* file according to usage and experiment requirement.

```
fileDirectory=/Users/fatimamariyam/IdeaProjects/file-system-pa1/src/data
port=8080
serverIP=127.0.0.1
serverPort=8080
maxDownloadAttempts=3
downloadDestinationDirectory=downloads
logDestinationDirectory=logs
numExperiments=5
```

- **fileDirectory:** Directory on the server side where from the files are to be fetched and downloaded.
- **serverIP:** server port
- **serverPort:** server port
- **maxDownloadAttempt:** maximum number of attempts for downloading the file.
- **downloadDestinationDirectory:** destination directory for downloaded files
- **logDestinationDirectory:** destination directory for logging files.
- **numExperiments:** Number of experiments

Execution

- Compile the code – run server – start client and enter the file names to downloaded:

```
/usr/bin/make -f /Users/fatimamariyam/IdeaProjects/file-system-pa1/src/Makefile -C /Users/fatimamariyam/IdeaProjects/file-system-pa1/src run_server
java Server_FS
/Users/fatimamariyam/IdeaProjects/file-system-pa1/src
Server started. Listening for connections...
Feb 17, 2024 2:23:07 AM Server_FS start
INFO: Server started. Listening for connections...
```

- For more than 1 file, select if files should be downloaded serially or parallelly.
- Files are downloaded at the configured download location (“*downloadDestinationDirectory*”)
- Logs with information(like durations) and error(if any) are generated at the configured logging directory. (“*logDestinationDirectory*”)

```

java Client_FS
get_files_list
Feb 17, 2024 2:23:22 AM Client_FS main
INFO: get_files_list request
Files available for download: [xyz.txt, abc.txt]
Enter comma-separated filenames to be downloaded:
xyz.txt, abc.txt
1: Serial Download
2: Parallel Download
1
Feb 17, 2024 2:23:33 AM Client_FS main
INFO: downloadFiles request
Downloaded: xyz.txt in : 5ms
Downloaded: abc.txt in : 1ms
Total download duration: 16ms
Feb 17, 2024 2:23:33 AM Client_FS downloadFile
INFO: Successfully downloaded file: xyz.txt in 5 ms
Feb 17, 2024 2:23:33 AM Client_FS downloadFile
INFO: Successfully downloaded file: abc.txt in 1 ms
Feb 17, 2024 2:23:33 AM Client_FS downloadFiles
INFO: Total download time: 16 ms

```

Serial Download

```

java Client_FS
get_files_list
Feb 17, 2024 2:24:02 AM Client_FS main
INFO: get_files_list request
Files available for download: [xyz.txt, abc.txt]
Enter comma-separated filenames to be downloaded:
xyz.txt, abc.txt
1: Serial Download
2: Parallel Download
2
Feb 17, 2024 2:24:09 AM Client_FS main
INFO: downloadFiles request
Feb 17, 2024 2:24:09 AM Client_FS downloadFiles
INFO: Total download time: 4 ms
Feb 17, 2024 2:24:09 AM Client_FS downloadFile
INFO: Successfully downloaded file: xyz.txt in 3 ms
Feb 17, 2024 2:24:09 AM Client_FS downloadFile
INFO: Successfully downloaded file: abc.txt in 3 ms
Total download duration: 4ms
Downloaded: xyz.txt in : 3ms
Downloaded: abc.txt in : 3ms

```

Parallel Download

Implementation:

The Java Socket API facilitates communication between the server and client. To enable node communication, both **FileInputStream** and **FileOutputStream** methods of the Socket API are utilized, which reads the file from server disk. Then **BufferedInputStream** for reading the file and sending the file to the client through Socket OutputStream.

On the client side, Socket InputStream reads the bytes sent from the server and through **BufferedOutputStream**, writes the file contents to the client disk.

- **Server_FS (Server logic)**

- The server stores a list of files available for access/download from one of its directories.
- The server also maintains a list of clients connecting concurrently for operations like getting a file list or downloading the file.
- Since the files are expected to be accessed concurrently via multiple clients, it must be thread safe and serializable.
- The server is run on the default port 8080, which can be configured through the config.properties file variable: *"port"*.
- Server IP is also configurable through the properties file variable *"serverIP"*.
- When the connection is established, the server continuously listens for any incoming connections from the client.
- The client request can either be *"get_files_list"* or *"download"*. Based on the request type, server takes the action and sends back the response.
- Once the server is disconnected, the socket connection disconnects.

- **Client_FS (Client logic)**

- The client connects with the server to request for a list of available files or to download a single file or multiple files.
- When requesting to download files, the client has the option to download a single file or more than one file from the server.
- If downloading more than one file, the client can download the file 1. Serially or 2. Parallel.
- The option to choose the download mechanism is provided after the client gives the list of files to be downloaded.
- The server IP and port is configured through the properties file. (default 127.0.0.1:8080)
- For the given problem experiments, multiple clients are connected concurrently to the server to request file downloads.
- In cases where multiple clients connect with the server, the resources should be thread-safe and serializable.
- Multiple clients affect the file download speeds which is mentioned as evaluated in the Performance document.
- The application experiments with downloading files of different sizes("128b" "512b" "2kb" "8kb" "32kb") by 4 concurrently running clients, with each client having 10 threads downloading the file of specified size.
- The experiments are repeated for 5 times(default). This is also configurable via *"numExperiments"* parameter in the properties file.